

UNIVERSIDAD NACIONAL DE JULIACA
FACULTAD DE CIENCIAS DE LA INGENIERÍA

ESCUELA PROFESIONAL
INGENIERÍA MECATRÓNICA



Proyecto #01
**Automatización para la mejora de la productividad con enfoque al
reciclaje neumático aplastador de latas ECO RECOMPENSA**

DOCENTE : Ing. Rojas Ccalla Ayrton

ESTUDIANTES : Correo Institucional

Arapa Uturunco Jose Miguel	2023108009.est@unaj.edu.pe
Benites Gallegos Will Jhon	2023108013.est@unaj.edu.pe
Condori Alvarez Willians	2023108018.est@unaj.edu.pe
Mamani Pampa Kevin Jesus	2023108011.est@unaj.edu.pe
Sihuacollo Vargas Alan	2022208032.est@unaj.edu.pe

ASIGNATURA : CIRCUITOS DIGITALES I

SEMESTRE : VII

FECHA : 03-05-2026

Juliaca - Perú
2026

Resumen

El presente trabajo desarrolla un sistema de automatización enfocado al reciclaje de latas de aluminio, denominado **ECO-RECOMPENSA**, que integra control electrónico, neumática industrial e interfaz gráfica digital con el fin de mejorar la productividad del proceso de reciclaje y generar un incentivo para el usuario. El sistema emplea un **Arduino Nano** como controlador principal, el cual gestiona dos pulsadores de mando (inicio y parada de emergencia), un módulo de relés de cuatro canales, un sensor ultrasónico y tres mini cilindros neumáticos Serie MAL accionados mediante válvulas solenoides 3/2 y 5/2 a 12 V. Una **Raspberry Pi 5** recibe señales del Arduino a través del puerto serial USB y ejecuta una interfaz gráfica desarrollada en Python que contabiliza en tiempo real las latas recicladas y genera un código de premio como incentivo al reciclaje responsable.

La secuencia de operación es completamente automática: al detectar una lata mediante el sensor ultrasónico, el sistema activa los relés en orden programado para empujar, posicionar y aplastar la lata, reduciendo su volumen para facilitar su almacenamiento y transporte al centro de reciclaje. El ciclo se repite en bucle mientras el sensor detecte latas presentes, y se detiene automáticamente al agotarse las mismas o al presionar el botón de emergencia.

Palabras clave: Arduino Nano, Raspberry Pi 5, neumática, válvulas solenoides, reciclaje automatizado, interfaz Python, ECO-RECOMPENSA.

Índice

1	Generalidades	1
1.1	Problema General	1
1.2	Problemas Específicos	1
1.3	Objetivo General	1
1.4	Objetivos Específicos	2
1.5	Justificación	2
2	Marco Teórico	2
2.1	Sistemas de Automatización Industrial	2
2.2	Microcontrolador Arduino Nano	3
2.3	Raspberry Pi 5	4
2.4	Neumática Industrial	5
2.4.1	Mini Cilindros Neumáticos Serie MAL	5
2.4.2	Válvulas Solenoides	6
2.4.3	Fórmulas Neumáticas Fundamentales	6
2.5	Módulo Relé de 4 Canales	7
2.6	Sensor Ultrasónico HC-SR04	8
2.7	Comunicación Serial UART	8
2.8	Interfaz Gráfica Python — Tkinter y Matplotlib	9
3	Materiales y Métodos	9
3.1	Materiales	9
3.2	Métodos	11
3.2.1	Esquema General del Sistema	11
3.2.2	Descripción del Subsistema de Control Electrónico	11
3.2.3	Descripción del Subsistema Neumático	12
3.2.4	Secuencia de Operación — Diagrama de Flujo	12
4	Programación	15
4.1	Código Arduino Nano	15
4.2	Interfaz Python en Raspberry Pi 5	18
5	Resultados	31
5.1	Pruebas del Subsistema Electrónico	31
5.2	Pruebas del Subsistema Neumático	31
5.3	Pruebas de la Interfaz Raspberry Pi 5	32
5.4	Rendimiento del Sistema	32
6	Discusión y Conclusiones	33

6.1	Discusión	33
6.2	Conclusiones	34
6.3	Recomendaciones	35
Anexos		37

1 Generalidades

1.1 Problema General

El reciclaje de latas de aluminio enfrenta un obstáculo logístico crítico: el elevado volumen que estas latas ocupan antes de ser transportadas al centro de reciclaje incrementa costos de almacenamiento y reduce la eficiencia del proceso. Además, la ausencia de sistemas automatizados accesibles y de bajo costo que incentiven al usuario a reciclar de forma continua limita el impacto ambiental de los programas de reciclaje existentes. En este contexto, se plantea la necesidad de diseñar un dispositivo que automatice el aplastado de latas, contabilice las unidades recicladas y recompense activamente al usuario.

1.2 Problemas Específicos

- ¿Cómo diseñar e implementar un sistema de control electrónico basado en Arduino Nano que gestione de forma secuencial y automática los actuadores neumáticos necesarios para el aplastado de latas?
- ¿Cómo fomentar hábitos sostenibles de reciclaje mediante la integración de tecnología de automatización y gamificación en un sistema ecológico inteligente?
- ¿Cómo establecer una comunicación serie confiable entre el Arduino Nano y la Raspberry Pi 5 para la transferencia de datos de reciclaje en tiempo real?
- ¿Cómo desarrollar en Python una interfaz gráfica que contabilice las latas recicladas, visualice estadísticas y genere códigos de recompensa para el usuario?
- ¿Cómo garantizar la seguridad del operario mediante un sistema de parada de emergencia que interrumpa el ciclo en cualquier instante?

1.3 Objetivo General

Diseñar, construir y validar un sistema automatizado de aplastado de latas de aluminio, denominado **ECO-RECOMPENSA**, que integre control electrónico con Arduino Nano, actuadores neumáticos industriales y una interfaz gráfica en Raspberry Pi 5, con el propósito de mejorar la productividad del proceso de reciclaje e incentivar su adopción mediante recompensas digitales.

1.4 Objetivos Específicos

- Implementar el circuito de control electrónico con Arduino Nano, módulo de relés de cuatro canales y los sensores necesarios para gestionar la secuencia de aplastado de latas.
- Diseñar e instalar el subsistema neumático con válvulas solenoides 3/2 y 5/2 a 12 V y tres mini cilindros Serie MAL que ejecuten las etapas de empuje y aplastado de forma coordinada.
- Programar la lógica de secuenciación, detección por sensor ultrasónico y comunicación serial para el funcionamiento autónomo en bucle del sistema.
- Desarrollar la interfaz gráfica Python en Raspberry Pi 5 con conteo de latas, visualización de estadísticas y generación automática de códigos de premio.
- Verificar el funcionamiento integral del sistema mediante pruebas de operación continua, validando la secuencia de relés, la detección del sensor y la comunicación con la Raspberry Pi 5.

1.5 Justificación

El aluminio es uno de los materiales más reciclables del planeta: reciclar una lata de aluminio requiere únicamente el 5 % de la energía necesaria para producir aluminio primario a partir de bauxita (Aluminum Association, 2022). Sin embargo, la ineficiencia logística en la etapa de recolección — principalmente por el gran volumen que ocupan las latas sin aplastar — desincentiva su reciclaje. El proyecto ECO-RECOMPENSA aborda simultáneamente tres dimensiones de esta problemática: *técnica* (automatización del aplastado), *logística* (reducción de volumen hasta en un 80 %) e *incentivo social* (recompensa digital por unidad reciclada). Su desarrollo como proyecto académico en la asignatura de Circuitos Digitales I permite además aplicar conceptos de lógica secuencial, comunicación serial y control de actuadores en un contexto de ingeniería real.

2 Marco Teórico

2.1 Sistemas de Automatización Industrial

La automatización industrial comprende el conjunto de técnicas y tecnologías orientadas a controlar procesos mecánicos, eléctricos o de manufactura con mínima intervención humana. Según [4], un sistema de automatización típico incluye cuatro componentes esenciales: actuadores que producen el trabajo físico, sensores que realimentan

el estado del proceso, un controlador que ejecuta la lógica de decisión y una interfaz hombre-máquina que permite supervisar y parametrizar el sistema.

En este proyecto, los tres primeros componentes residen en el subsistema Arduino Nano—relés—neumática, mientras que la interfaz gráfica en Raspberry Pi 5 cumple la función de supervisión y registro.

2.2 Microcontrolador Arduino Nano

El Arduino Nano es una plataforma de desarrollo compacta basada en el microcontrolador **ATmega328P** fabricado por Atmel (ahora Microchip). Sus características principales son:

- Microcontrolador: ATmega328P a 16 MHz.
- Voltaje de operación: 5 V.
- 14 pines de E/S digitales (6 con salida PWM).
- 8 entradas analógicas de 10 bits (resolución: $\frac{5\text{ V}}{1023} \approx 4,88\text{ mV/bit}$).
- Memoria Flash: 32 KB; SRAM: 2 KB; EEPROM: 1 KB.
- Interfaz USB-Serie via chip CH340 o FT232.
- Corriente máxima por pin de E/S: 40 mA.

El Arduino Nano fue seleccionado para este proyecto por su reducido tamaño, bajo consumo y capacidad suficiente para gestionar la lógica de secuenciación, la lectura del sensor ultrasónico y la comunicación serial con la Raspberry Pi 5.

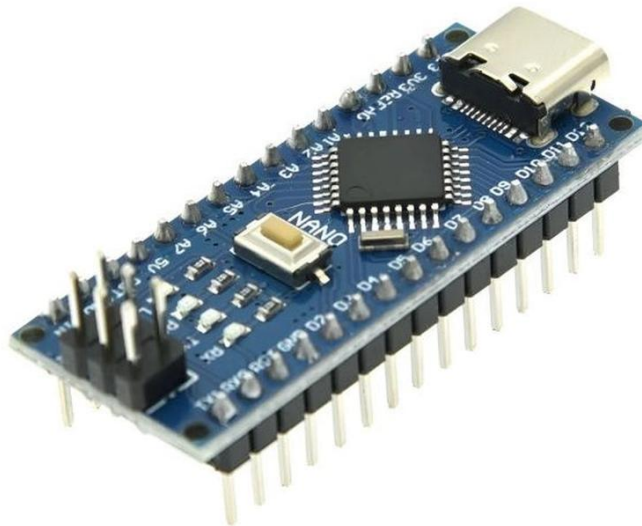


Figura 1: Microcontrolador arduino nano

2.3 Raspberry Pi 5

La Raspberry Pi 5 es una computadora monoplaca de alto rendimiento lanzada en 2023. Incorpora un procesador **Arm Cortex-A76** de cuatro núcleos a 2.4 GHz, hasta 8 GB de RAM LPDDR4X, salida micro-HDMI de hasta 4K, puertos USB 3.0, interfaz PCIe 2.0 y múltiples pines GPIO. Su capacidad de ejecutar Raspberry Pi OS (basado en Debian) la convierte en la plataforma idónea para correr aplicaciones Python con librerías gráficas avanzadas como Tkinter y Matplotlib.

En el contexto de este proyecto, la Raspberry Pi 5 actúa como unidad de supervisión: recibe los datos del Arduino Nano por puerto serial USB, los procesa en tiempo real y actualiza la interfaz gráfica con el conteo de latas recicladas y los códigos de recompensa generados.

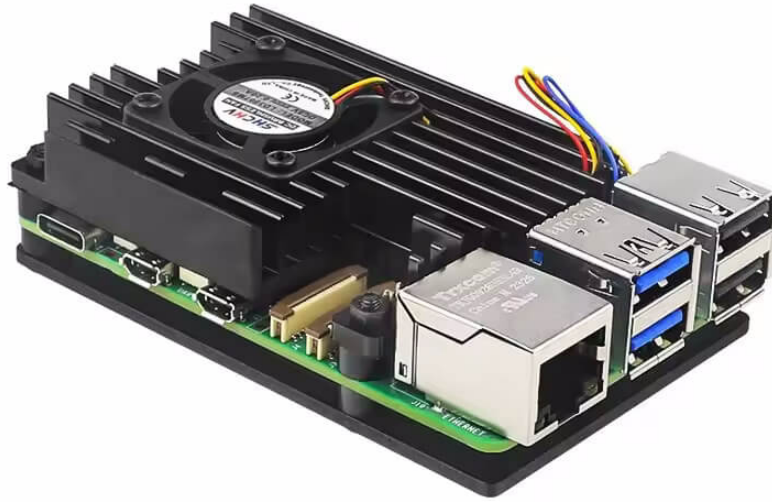


Figura 2: Raspberry pi 5

2.4 Neumática Industrial

La neumática es la rama de la física e ingeniería que estudia el empleo del aire comprimido como fluido de trabajo para transmitir fuerza y movimiento. Las ventajas del control neumático frente a otros sistemas de actuación son la limpieza del fluido de trabajo, la alta velocidad de respuesta, la capacidad de sobrecarga sin daño permanente y el bajo costo de mantenimiento [3].

2.4.1 Mini Cilindros Neumáticos Serie MAL

Los mini cilindros Serie MAL (conforme a norma ISO 6432) son actuadores de doble efecto de pequeño formato, con diámetros de 10 a 50 mm y carreras típicas de 10 a 200 mm. En este proyecto se emplean tres unidades:

N°	Denominación	Función	Válvula asociada
1	Cilindro empujador	Empuje lateral de la lata	Válvula 3/2 (relé 1 y 2)
2	Cilindro aplastador A	Aplastado de la lata	Válvula 5/2 (relé 3)
3	Cilindro aplastador B	Aplastado de la lata	Válvula 5/2 (relé 3)

2.4.2 Válvulas Solenoides

Válvula 3/2 (tres vías, dos posiciones): controla el avance y el retroceso de un único cilindro. Su accionamiento es por solenoide a 12 V DC con retorno por muelle. En este proyecto se emplean dos válvulas de este tipo:

- *Válvula 3/2 — avance*: alimenta el cilindro empujador para que avance y empuje la lata hacia la zona de caída.
- *Válvula 3/2 — retroceso*: revierte el circuito neumático para que el cilindro empujador regrese a su posición inicial.

Válvula 5/2 (cinco vías, dos posiciones): permite distribuir el flujo de aire a dos actuadores simultáneos. Al ser energizada por el relé 2, comparte la presión entre los cilindros 2 y 3 para el aplastado sincrónico de la lata.



Figura 3: Diferencia entre Valvulas 3/2 y valvulas 5/2

2.4.3 Fórmulas Neumáticas Fundamentales

La fuerza de avance de un cilindro de doble efecto se calcula como:

$$F = P \cdot A = P \cdot \frac{\pi D^2}{4} \quad (1)$$

donde F es la fuerza de empuje (N), P es la presión de trabajo (Pa) y D el diámetro interior del cilindro (m).

Para una presión de trabajo de $P = 5 \text{ bar} = 5 \times 10^5 \text{ Pa}$ y un cilindro de $D = 20 \text{ mm}$:

$$F = 5 \times 10^5 \cdot \frac{\pi(0,020)^2}{4} \approx 157 \text{ N} \quad (2)$$

Esta fuerza es más que suficiente para aplastar una lata de aluminio estándar, cuya resistencia al aplastado es de aproximadamente 20 a 50 N.

2.5 Módulo Relé de 4 Canales

Un relé es un dispositivo electromecánico de conmutación que permite controlar cargas de alta tensión/corriente mediante señales de control de bajo nivel. El módulo de cuatro canales empleado incorpora:

- **Optoacopladores:** aíslan galvánicamente el circuito de control (5 V del Arduino) del circuito de potencia (12 V DC de los solenoides), protegiendo el microcontrolador ante picos de tensión inductivos generados por las bobinas de las válvulas.
- **Transistores de disparo:** amplifican la corriente de control para accionar la bobina del relé electromagnético.
- **Contactos NA/NC:** cada canal dispone de contacto normalmente abierto (NA) y normalmente cerrado (NC), permitiendo configurar la lógica de activación según la aplicación.

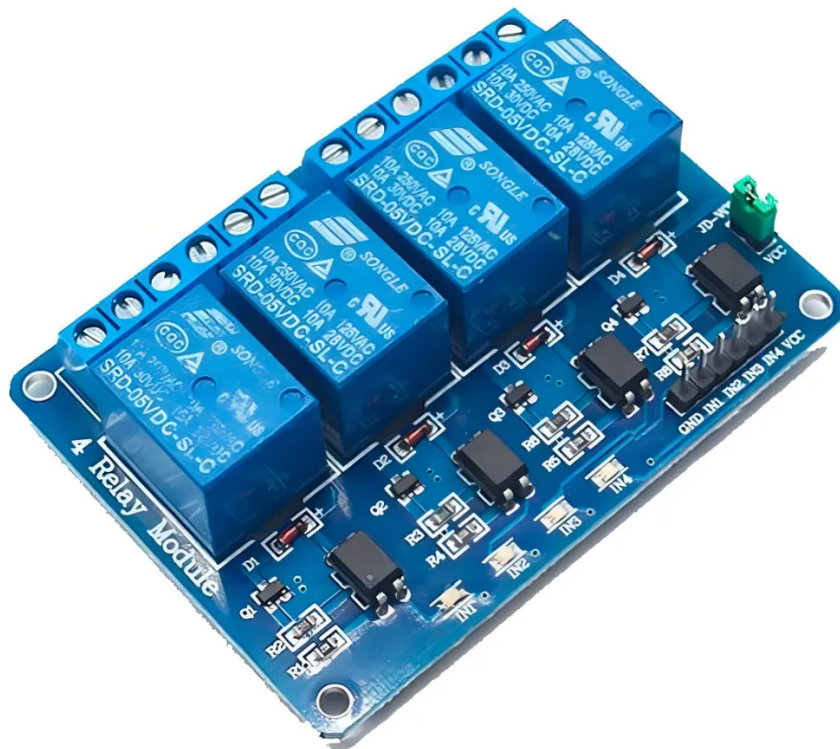


Figura 4: Modulo Rele de 4 canales

Los tres canales utilizados y sus funciones se resumen en la Tabla 1.

Relé	Pin Arduino	Carga controlada	Acción
1	D6	Válvula solenoide 3/2	Avance del cilindro empujador (C1)
2	D7	Válvula solenoide 3/2	Retroceso del cilindro empujador (C1)
3	D8	Válvula solenoide 5/2	Aplastado simultáneo de cilindros C2 y C3
—	—	No utilizado	Reserva del sistema

Cuadro 1: Asignación de relés del sistema neumático

2.6 Sensor Ultrasónico HC-SR04

El sensor ultrasónico HC-SR04 opera emitiendo pulsos de sonido a 40 kHz a través del pin TRIG y midiendo el tiempo que tarda el eco en regresar al pin ECHO. La distancia se calcula mediante:

$$d = \frac{t_{\text{echo}} \times v_{\text{sonido}}}{2} = \frac{t_{\text{echo}} \times 343}{2} \quad (3)$$

donde d es la distancia (m), t_{echo} el tiempo de vuelo del eco (s) y $v_{\text{sonido}} = 343 \text{ m/s}$ la velocidad del sonido en aire a 20 °C.

Sus especificaciones principales son:

- Rango de medición: 2 cm a 400 cm.
- Precisión: $\pm 3 \text{ mm}$.
- Ángulo de cobertura efectivo: 15°.
- Voltaje de operación: 5 V DC; corriente: 15 mA.

En este proyecto el sensor se posiciona en la zona de aplastado. Cuando detecta una distancia menor al umbral configurado ($d < d_{\text{umbral}}$), confirma la presencia de una lata y habilita la activación del relé 2.

2.7 Comunicación Serial UART

El protocolo UART (*Universal Asynchronous Receiver/Transmitter*) es un estándar de comunicación serie asíncrona ampliamente utilizado. La velocidad de comunicación se expresa en baudios (bits por segundo). En este proyecto se usa una velocidad de **9600 bps**, que ofrece un equilibrio óptimo entre velocidad de transmisión y robustez ante interferencias en el cableado USB.

El Arduino Nano envía la cadena `RECICLADA\n"` por el puerto serial cada vez que una lata es procesada. La Raspberry Pi 5 ejecuta un hilo de escucha permanente que

detecta esta cadena e incrementa el contador de la interfaz gráfica.

2.8 Interfaz Gráfica Python — Tkinter y Matplotlib

La interfaz gráfica se desarrolló en Python 3 empleando las siguientes librerías:

- **tkinter**: biblioteca gráfica estándar de Python para construir interfaces de escritorio. Proporciona widgets como botones, etiquetas, barras de progreso y lienzos.
- **serial** (PySerial): librería para la comunicación con puertos series, permite leer y escribir datos del puerto USB conectado al Arduino.
- **matplotlib**: librería de visualización científica que permite generar gráficas de barras, líneas e histogramas directamente embebidos en la ventana Tkinter.
- **random** y **string**: usados para generar los códigos alfanuméricos de premio de forma pseudoaleatoria.
- **threading**: permite ejecutar la lectura serial en un hilo separado para no bloquear la interfaz gráfica.

3 Materiales y Métodos

3.1 Materiales

Los materiales y componentes empleados en la construcción del sistema ECO-RECOMPENSA se detallan en la Tabla 2.

Cuadro 2: Lista completa de materiales del proyecto ECO-RECOMPENSA

N°	Componente	Especificación	Función en el sistema	Cant.
1	Arduino Nano	ATmega328P, 5 V, 16 MHz	Controlador principal del sistema	1
2	Raspberry Pi 5	Cortex-A76, hasta 8 GB RAM	Interfaz gráfica y conteo de reciclaje	1
3	Pantalla HDMI	Monitor con entrada HDMI	Visualización de interfaz Python	1
4	Módulo relé 4 canales	5 V trigger, 250 VAC/10 A	Activar solenoides de las válvulas	1
5	Sensor ultrasónico	HC-SR04, rango 2–400 cm	Detectar presencia de lata	1
6	Pulsador START	NA momentáneo, verde	Inicio del ciclo automático	1
7	Pulsador STOP Emergencia	NC momentáneo, rojo	Parada de emergencia del sistema	1
8	Válvula solenoide 3/2	12 V DC, retorno muelle	Control avance cilindro empujador	1
9	Válvula solenoide 3/2	12 V DC, retorno muelle	Control retroceso cilindro empujador	1
10	Válvula solenoide 5/2	12 V DC, doble solenoide	Control simultáneo cilindros 2 y 3	1
11	Mini cilindro MAL	Serie MAL, doble efecto, ISO 6432	Empuje lateral de la lata	1
12	Mini cilindro MAL	Serie MAL, doble efecto, ISO 6432	Aplastado de la lata (par sincrónico)	2
13	Mangueras neumáticas	Ø4–6 mm, PU/nylon	Conducción de aire comprimido	c/n
14	Racores neumáticos	Push-in Ø4–6 mm	Conexión de mangueras	c/n
15	Fuente 12 V DC	2 A mínimo	Alimentación de solenoides y relés	1
16	Cable USB A–miniUSB	Arduino Nano ↔ RPi5	Comunicación serial y alimentación	1
17	Cableado AWG 22	Rojo/negro/varios colores	Circuito eléctrico de control	c/n
18	Protoboard/PCB	830 puntos o PCB perforada	Montaje del circuito de control	1
19	Estructura mecánica	Perfil aluminio o MDF	Soporte de cilindros y lata	1
20	Tacho de almacenamiento	Recipiente mediano	Almacenamiento de latas aplastadas	1

3.2 Métodos

3.2.1 Esquema General del Sistema

El sistema ECO-RECOMPENSA se divide en tres subsistemas interconectados que operan de forma coordinada:

1. **Subsistema de control electrónico:** Arduino Nano + módulo relé + sensores.
2. **Subsistema neumático:** válvulas solenoides + cilindros MAL.
3. **Subsistema de monitoreo:** Raspberry Pi 5 + pantalla HDMI + interfaz Python.

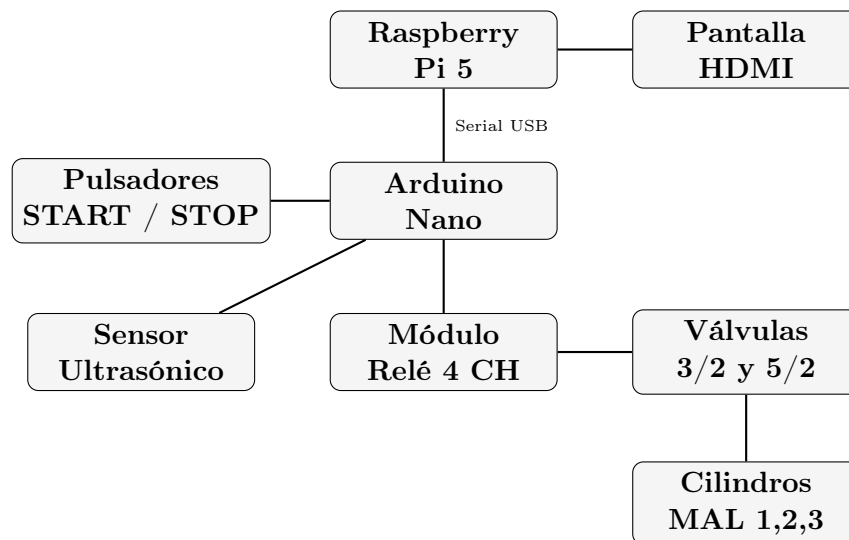


Figura 5: Diagrama de bloques del sistema ECO-RECOMPENSA

3.2.2 Descripción del Subsistema de Control Electrónico

El Arduino Nano actúa como cerebro del sistema. En su pin D2 se conecta el pulsador START (con resistencia pull-up interna); en el pin D3, el pulsador STOP de emergencia (configurado como NC con interrupción de alta prioridad). El sensor ultrasónico se conecta con TRIG en D9 y ECHO en D10. Los relés se activan con salidas de los pines D5, D6 y D7 respectivamente. La lógica de control es **activa en BAJO** dado que el módulo relé dispone de optoacopladores de entrada invertida.

Los pulsadores operan de la siguiente manera:

- **START (verde):** al pulsarlo el sistema entra en modo de operación continua. La secuencia de relés se ejecuta cíclicamente mientras el sensor detecte latas.
- **STOP de emergencia (rojo):** interrumpe inmediatamente el ciclo en cualquier punto, desactivando todos los relés y llevando el sistema al estado seguro (válvulas sin presión, cilindros en posición de retroceso).

3.2.3 Descripción del Subsistema Neumático

El compresor suministra aire a una presión de trabajo de 4 a 6 bar. Las mangueras de poliuretano de $\varnothing 4-6$ mm conectan el compresor a las válvulas solenoides y estas a los cilindros. La distribución de presión es la siguiente:

1. El **relé 1** energiza la válvula 3/2-avance, que permite el paso de aire al puerto de avance del cilindro 1. El vástago avanza y empuja la lata hacia la rampa de caída.
2. El **relé 2** energiza la válvula 3/2-retroceso, que revierte el circuito del cilindro 1. El vástago retrocede a su posición inicial.
3. El **relé 3** energiza la válvula 5/2, que distribuye simultáneamente aire a los cilindros 2 y 3. Ambos vástagos avanzan al mismo tiempo aplastando la lata desde dos direcciones opuestas o verticales, dependiendo del diseño mecánico adoptado. La lata aplastada cae por el orificio inferior al tacho de almacenamiento.

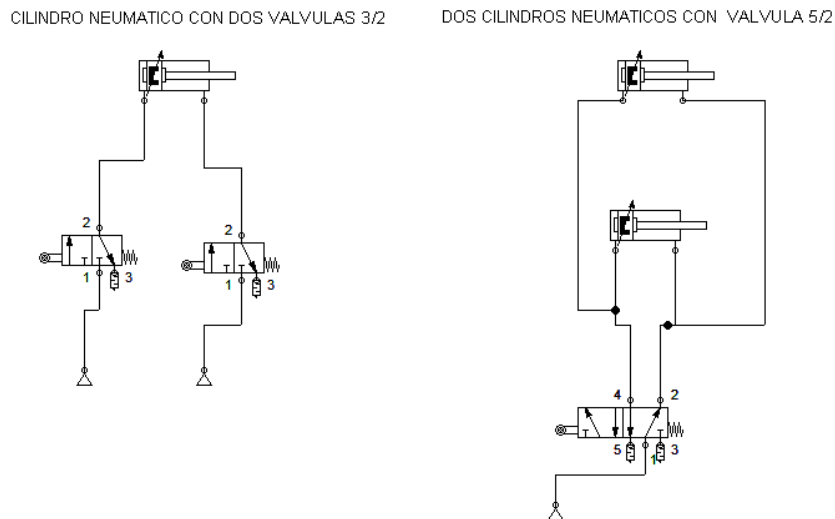


Figura 6: Esquema neumático del sistema ECO-RECOMPENSA

3.2.4 Secuencia de Operación — Diagrama de Flujo

La lógica de operación del sistema es la siguiente:

1. El operario deposita latas en la bandeja superior y presiona **START**. El sistema se habilita y entra en estado activo.
2. Tras un retardo inicial ($t = 3000$ ms), el Arduino activa el **relé 1**, energizando la válvula 3/2 de avance, provocando la extensión del cilindro 1.

3. Finalizado el tiempo de activación, el relé 1 se desactiva.
4. El Arduino activa el **relé 2**, energizando la válvula 3/2 de retroceso, provocando la retracción del cilindro 1.
5. Finalizado el tiempo de activación, el relé 2 se desactiva.
6. La lata cae por gravedad hacia la zona de detección.
7. El sensor ultrasónico monitorea continuamente la presencia de la lata. Cuando se detecta, el sistema continúa con la etapa de aplastado.
8. El Arduino activa el **relé 3**, energizando la válvula 5/2, provocando el accionamiento simultáneo de los cilindros 2 y 3.
9. El relé 3 ejecuta múltiples ciclos ON/OFF (4 repeticiones) para realizar el aplastado completo de la lata.
10. Al finalizar el proceso, el Arduino envía la cadena RECICLADA" por el puerto serial.
11. La Raspberry Pi 5 recibe la señal, incrementa el contador y actualiza la interfaz del sistema.
12. El sistema reinicia automáticamente el ciclo desde el relé 1 mientras permanezca activo y continúe la operación.
13. Si el sensor no detecta ninguna lata, el sistema permanece en estado de espera dentro del ciclo.
14. En cualquier instante, al presionar **STOP**, todos los relés se desactivan inmediatamente y el sistema se detiene por completo.

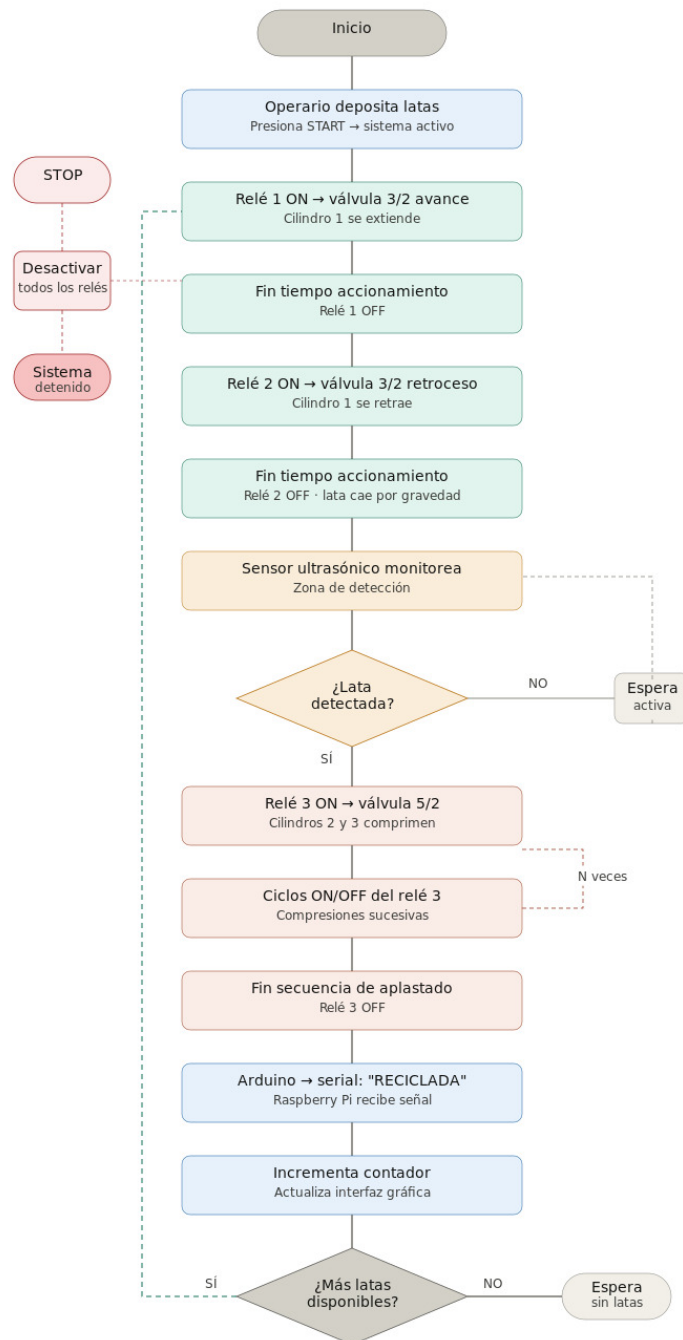


Figura 7: Diagrama de flujo de la lógica de control del Arduino Nano

4 Programación

4.1 Código Arduino Nano

El programa cargado en el Arduino Nano gestiona la secuencia completa de aplastado, la lectura del sensor ultrasónico, el control de los relés y la comunicación serial con la Raspberry Pi 5.

```
1 #define TRIG 2
2 #define ECHO 3
3
4 #define START_BTN 4
5 #define STOP_BTN 5
6
7 #define RELAY1 6
8 #define RELAY2 7
9 #define RELAY3 8
10
11 bool sistemaActivo = false;
12
13 void apagarTodo() {
14     digitalWrite(RELAY1, HIGH);
15     digitalWrite(RELAY2, HIGH);
16     digitalWrite(RELAY3, HIGH);
17 }
18
19 void verificarStop() {
20     if (digitalRead(STOP_BTN) == LOW) {
21         sistemaActivo = false;
22         apagarTodo();
23     }
24 }
25
26 long medirDistancia() {
27     digitalWrite(TRIG, LOW);
28     delayMicroseconds(5);
29
30     digitalWrite(TRIG, HIGH);
31     delayMicroseconds(10);
32
33     digitalWrite(TRIG, LOW);
34
35     long duracion = pulseIn(ECHO, HIGH, 50000);
36
37     if (duracion <= 0) return -1;
38 }
```

```

39     return duracion / 58;
40 }
41
42 void secuenciaRele1() {
43     digitalWrite(RELAY1, LOW);
44
45     unsigned long t = millis();
46
47     while (millis() - t < 1500) {
48         verificarStop();
49         if (!sistemaActivo) return;
50     }
51
52     digitalWrite(RELAY1, HIGH);
53 }
54
55 void secuenciaRele2() {
56     digitalWrite(RELAY2, LOW);
57
58     unsigned long t = millis();
59
60     while (millis() - t < 1500) {
61         verificarStop();
62         if (!sistemaActivo) return;
63     }
64
65     digitalWrite(RELAY2, HIGH);
66 }
67
68 void secuenciaRele3() {
69     for (int i = 0; i < 4; i++) {
70
71         digitalWrite(RELAY3, LOW);
72
73         unsigned long t = millis();
74
75         while (millis() - t < 1500) {
76             verificarStop();
77             if (!sistemaActivo) return;
78         }
79
80         digitalWrite(RELAY3, HIGH);
81
82         t = millis();
83
84         while (millis() - t < 1500) {
85             verificarStop();

```

```

86     if (!sistemaActivo) return;
87 }
88 }
89 }
90
91 void setup() {
92     pinMode(TRIG, OUTPUT);
93     pinMode(ECHO, INPUT);
94
95     pinMode(START_BTN, INPUT_PULLUP);
96     pinMode(STOP_BTN, INPUT_PULLUP);
97
98     pinMode(RELAY1, OUTPUT);
99     pinMode(RELAY2, OUTPUT);
100    pinMode(RELAY3, OUTPUT);
101
102    apagarTodo();
103
104    Serial.begin(9600);
105 }
106
107 void loop() {
108     verificarStop();
109
110     if (digitalRead(START_BTN) == LOW && !sistemaActivo) {
111         sistemaActivo = true;
112         delay(3000);
113     }
114
115     if (!sistemaActivo) return;
116
117     secuenciaRele1();
118     if (!sistemaActivo) return;
119
120     secuenciaRele2();
121     if (!sistemaActivo) return;
122
123     bool objetoDetectado = false;
124
125     while (sistemaActivo) {
126         verificarStop();
127
128         long distancia = medirDistancia();
129
130         if (distancia > 0 && distancia <= 10) {
131             objetoDetectado = true;
132             break;

```

```

133     }
134
135     delay(100);
136 }
137
138 if (objetoDetectado && sistemaActivo) {
139     secuenciaRele3();
140
141     if (sistemaActivo) {
142         Serial.println("RECICLADA");
143         sistemaActivo = false;
144         apagarTodo();
145     }
146 }
147
148 delay(500);
149 }

```

Listing 1: Código completo del Arduino Nano — ECO-RECOMPENSA

4.2 Interfaz Python en Raspberry Pi 5

El script Python se ejecuta automáticamente al arrancar la Raspberry Pi 5. Lee el puerto serial, contabiliza las latas recicladas y actualiza la interfaz gráfica Tkinter en tiempo real.

```

1 import tkinter as tk
2 import random
3 import string
4 from PIL import Image, ImageTk
5 from tkinter import ttk
6 import serial
7 import threading
8 import time
9
10 # =====
11 # CONEXION ARDUINO
12 # =====
13
14 try:
15     arduino = serial.Serial('/dev/ttyUSB0', 9600, timeout=1)
16     time.sleep(2)
17     print("Arduino conectado")
18 except Exception as e:
19     print("Error conectando Arduino:", e)
20     arduino = None

```

```

21
22 # =====
23 # PALETA DE COLORES
24 # =====
25
26 C_BG          = "#0D1F0F"
27 C_PANEL       = "#102613"
28 C_CARD        = "#152E17"
29 C_SHADOW      = "#050E06"
30 C_GREEN1      = "#00E676"
31 C_GREEN2      = "#69F0AE"
32 C_GOLD        = "#FFD600"
33 C_ACCENT      = "#B9F6CA"
34 C_DARK_TEXT   = "#0D1F0F"
35 C_RED_WARN    = "#FF6D00"
36 C_BORDER      = "#1B5E20"
37 C_HEADER      = "#0A2E0C"
38 C_QR_BG       = "#0D2510"
39
40 # =====
41 # VARIABLES GLOBALES
42 # =====
43
44 contador      = 0
45 ultimo_tiempo = 0
46 _img_refs     = {}    # evitar garbage collection
47
48 # =====
49 # VENTANA PRINCIPAL
50 # =====
51
52 root = tk.Tk()
53 root.title("ECO RECOMPENSA      Sistema Inteligente de Reciclaje")
54 root.configure(bg=C_BG)
55 root.resizable(True, True)
56
57 root.update_idletasks()
58 try:
59     root.attributes("-zoomed", True)    # Linux / Raspberry Pi
60 except Exception:
61     root.state("zoomed")                # Windows (fallback)
62
63 # =====
64 # CANVAS PRINCIPAL
65 # =====
66
67 cv = tk.Canvas(root, bg=C_BG, highlightthickness=0)

```

```

68 cv.pack(fill="both", expand=True)
69
70 # =====
71 # VARIABLES TKINTER
72 # =====
73
74 contador_var = tk.StringVar(value="0")
75 contador2_var = tk.StringVar(value="0")
76 codigo_var = tk.StringVar(value="??? - ???")
77
78 # =====
79 # HELPERS
80 # =====
81
82 def W():
83     return cv.winfo_width()
84
85 def H():
86     return cv.winfo_height()
87
88 def fs(base, ref=1280):
89     """Tama o de fuente escalado al ancho real de la ventana."""
90     return max(8, int(base * W() / ref))
91
92 def rect(x, y, w, h, fill=C_CARD, border=C_GREEN1, bw=2,
93         shadow=True, tag="layout"):
94     """Rect ngulo con sombra opcional dibujado en el canvas."""
95     if shadow:
96         cv.create_rectangle(x+4, y+4, x+w+4, y+h+4,
97                             fill=C_SHADOW, outline="", tags=tag)
98         cv.create_rectangle(x, y, x+w, y+h,
99                             fill=fill, outline=border, width=bw, tags=tag)
100
101 def place(widget, x, y, w, h, **cfg):
102     """Configura y posiciona un widget Tk sobre el canvas."""
103     if cfg:
104         try:
105             widget.config(**cfg)
106         except tk.TclError:
107             pass
108     widget.place(x=int(x), y=int(y), width=int(w), height=int(h))
109     widget.lift()
110
111 # =====
112 # CREAR WIDGETS TK (una sola vez)
113 # =====
114

```

```

115 def mk(text="", font=None, bg=C_CARD, fg=C_GREEN1,
116         anchor="center", wraplength=0, textvariable=None):
117     kw = dict(bg=bg, fg=fg, anchor=anchor,
118              font=font or ("Courier New", 12, "bold"))
119     if wraplength:
120         kw["wraplength"] = wraplength
121     if textvariable is not None:
122         kw["textvariable"] = textvariable
123     else:
124         kw["text"] = text
125     return tk.Label(root, **kw)
126
127 # Panel izquierdo
128 lbl_contador = mk(textvariable=contador_var,
129                   font=("Courier New", 72, "bold"), bg=C_CARD, fg=
130                     C_GREEN1)
131 lbl_estado = mk("Esperando reciclaje...",
132                 bg="#1A3D1C", fg=C_RED_WARN, wraplength=400)
133 lbl_pct = mk("0%", bg=C_PANEL, fg=C_GREEN1)
134
135 # Panel derecho
136 lbl_eco_brand = mk("ECO RECOMPENSA", bg=C_PANEL, fg=C_GOLD)
137 lbl_cod_titulo = mk("C DIGO DE PREMIO", bg=C_QR_BG, fg=C_GOLD)
138 lbl_codigo = mk(textvariable=codigo_var,
139                 font=("Courier New", 36, "bold"), bg=C_QR_BG, fg=
140                     C_GOLD)
141 lbl_qr_info = mk("Escanea el QR para conocer m s sobre el proyecto",
142                 bg=C_PANEL, fg=C_GREEN2)
143 lbl_pasos = mk("Inserta la lata Recibe tu c digo",
144               "Gracias por reciclar!",
145               bg=C_PANEL, fg=C_ACCENT)
146 lbl_total_t = mk("TOTAL RECICLADAS HOY", bg=C_CARD, fg=C_GREEN2)
147 lbl_total_v = mk(textvariable=contador2_var,
148                 font=("Courier New", 32, "bold"), bg=C_CARD, fg=
149                     C_GREEN1)
150 lbl_info1 = mk("Cada lata hace la diferencia", bg=C_PANEL, fg=
151               C_GREEN2)
152 lbl_info2 = mk("Juntos por el planeta", bg=C_PANEL, fg=
153               C_GREEN2)
154 lbl_info3 = mk("M s info en el QR", bg=C_PANEL, fg=
155               C_GOLD)
156
157 lbl_footer = mk(
158     "ECO RECOMPENSA Sistema de Reciclaje Inteligente",
159     "Raspberry Pi 5",
160     bg="#071008", fg=C_GREEN2)

```

```

154 # Barra de progreso
155 style = ttk.Style()
156 style.theme_use("clam")
157 style.configure("Eco.Horizontal.TProgressbar",
158                 troughcolor=C_CARD, background=C_GREEN1,
159                 bordercolor=C_GREEN1, lightcolor=C_GREEN2,
160                 darkcolor="#004D20", thickness=20)
161 progreso = ttk.Progressbar(root, style="Eco.Horizontal.TProgressbar",
162                             orient="horizontal", mode="determinate")
163
164 # Historial
165 historial = tk.Listbox(
166     root, font=("Courier New", 10),
167     bg="#0B1E0D", fg=C_GREEN2,
168     selectbackground=C_GREEN1, selectforeground=C_DARK_TEXT,
169     relief="flat", borderwidth=0,
170     highlightthickness=1, highlightcolor=C_GREEN1)
171
172 # =====
173 # CARGA DE IM GENES (cach por tama o)
174 # =====
175
176 def _load_img(path, size, key):
177     """Carga y redimensiona una imagen; usa cach si el tama o no
178     cambi ."""
179     sz = max(30, size)
180     if _img_refs.get(key + "_sz") == sz:
181         return _img_refs.get(key), sz
182     try:
183         img = Image.open(path).convert("RGBA")
184         img = img.resize((sz, sz), Image.LANCZOS)
185         ph = ImageTk.PhotoImage(img)
186         _img_refs[key] = ph
187         _img_refs[key + "_sz"] = sz
188         return ph, sz
189     except Exception:
190         return None, sz
191
192 def _load_qr(size):
193     """Carga el QR con borde verde decorativo."""
194     sz = max(60, size)
195     if _img_refs.get("qr_sz") == sz:
196         return _img_refs.get("qr"), sz
197     try:
198         img = Image.open("codigo.PNG").convert("RGBA")
199         b = Image.new("RGBA", (img.width + 16, img.height + 16), (0,
200         230, 118, 255))

```

```

199     b.paste(img, (8, 8))
200     b = b.resize((sz, sz), Image.LANCZOS)
201     ph = ImageTk.PhotoImage(b)
202     _img_refs["qr"] = ph
203     _img_refs["qr_sz"] = sz
204     return ph, sz
205 except Exception:
206     return None, sz
207
208 # =====
209 # REDIBUJADO RESPONSIVO
210 # =====
211
212 def redibujar(event=None):
213     w, h = W(), H()
214     if w < 50 or h < 50:
215         return
216
217     cv.delete("layout")
218
219     PAD = max(8, int(w * 0.012))
220     HDR_H = max(55, int(h * 0.13))
221     FTR_H = max(22, int(h * 0.038))
222     BY = HDR_H + PAD # inicio del cuerpo
223     BH = h - HDR_H - FTR_H - PAD * 2 # alto del cuerpo
224
225     # Columnas 31% / 22% / resto
226     C1W = int(w * 0.31)
227     C2W = int(w * 0.22)
228     C3W = w - C1W - C2W - PAD * 4
229     C1X = PAD
230     C2X = C1X + C1W + PAD
231     C3X = C2X + C2W + PAD
232
233     # ENCABEZADO
234
235     rect(PAD, PAD, w - PAD * 2, HDR_H - PAD,
236         fill=C_HEADER, border=C_GREEN1)
237
238     logo_sz = max(40, int(HDR_H * 0.78))
239     logo_ph, logo_sz = _load_img("logo.png", logo_sz, "logo_small")
240     logo_x = PAD * 2 + logo_sz // 2 + 4
241     logo_y = PAD + (HDR_H - PAD) // 2
242     if logo_ph:
243         cv.create_image(logo_x, logo_y, image=logo_ph,
244             anchor="center", tags="layout")

```

```

244     else:
245         cv.create_text(logo_x, logo_y, text="    ",
246                        font=("Courier New", fs(32), "bold"),
247                        fill=C_GREEN1, anchor="center", tags="layout")
248
249     cx = w // 2
250     cv.create_text(cx, PAD + int((HDR_H - PAD) * 0.32),
251                   text="SISTEMA AUTOMATIZADO DE RECICLAJE",
252                   font=("Courier New", fs(22), "bold"),
253                   fill=C_GREEN1, anchor="center", tags="layout")
254     cv.create_text(cx, PAD + int((HDR_H - PAD) * 0.70),
255                   text="ECO RECOMPENSA          RECICLA Y GANA",
256                   font=("Courier New", fs(12), "bold"),
257                   fill=C_GREEN2, anchor="center", tags="layout")
258
259     cv.create_line(PAD * 2, HDR_H, w - PAD * 2, HDR_H,
260                   fill=C_GREEN1, width=1, dash=(6, 4), tags="layout")
261
262     #          PANEL IZQUIERDO
263
264     rect(C1X, BY, C1W, BH, fill=C_PANEL, border=C_GREEN1)
265
266     sp = PAD * 2    # padding interno del panel
267     sy = BY + sp
268
269     # T tulo "LATAS RECICLADAS"
270     tit_h = int(h * 0.040)
271     cv.create_text(C1X + C1W // 2, sy + tit_h // 2,
272                   text="LATAS RECICLADAS",
273                   font=("Courier New", fs(13), "bold"),
274                   fill=C_GREEN2, anchor="center", tags="layout")
275
276     sy += tit_h + PAD
277
278     # Caja del contador
279     cnt_h = int(BH * 0.20)
280     rect(C1X + sp, sy, C1W - sp * 2, cnt_h, fill=C_CARD, border=C_GREEN2)
281     place(lbl_contador, C1X + sp, sy, C1W - sp * 2, cnt_h,
282           font=("Courier New", fs(56), "bold"), bg=C_CARD, fg=C_GREEN1)
283
284     sy += cnt_h + PAD
285
286     # Estado
287     est_h = int(h * 0.070)
288     place(lbl_estado, C1X + sp, sy, C1W - sp * 2, est_h,
289           font=("Courier New", fs(11), "bold"),
290           wraplength=C1W - sp * 4)
291
292     sy += est_h + PAD

```

```

289
290 cv.create_line(C1X + sp, sy, C1X + C1W - sp, sy,
291               fill=C_BORDER, dash=(4, 4), tags="layout")
292 sy += PAD
293
294 # Nivel ecológico + barra
295 cv.create_text(C1X + C1W // 2, sy + int(h * 0.018),
296               text="NIVEL ECOLÓGICO",
297               font=("Courier New", fs(10), "bold"),
298               fill=C_GREEN2, anchor="center", tags="layout")
299 sy += int(h * 0.038)
300
301 bar_h = max(14, int(h * 0.030))
302 place(progreso, C1X + sp, sy, C1W - sp * 2, bar_h)
303 sy += bar_h + 4
304
305 place(lbl_pct, C1X + C1W // 2 - 20, sy, 60, int(h * 0.030),
306       font=("Courier New", fs(10), "bold"), bg=C_PANEL, fg=C_GREEN1)
307 sy += int(h * 0.030) + PAD
308
309 cv.create_line(C1X + sp, sy, C1X + C1W - sp, sy,
310               fill=C_BORDER, dash=(4, 4), tags="layout")
311 sy += PAD
312
313 # Historial
314 cv.create_text(C1X + C1W // 2, sy + int(h * 0.018),
315               text="    HISTORIAL DE LATAS",
316               font=("Courier New", fs(10), "bold"),
317               fill=C_GREEN2, anchor="center", tags="layout")
318 sy += int(h * 0.038)
319
320 hist_h = BY + BH - sy - PAD
321 if hist_h > 20:
322     place(historial, C1X + sp, sy, C1W - sp * 2, hist_h,
323           font=("Courier New", fs(9)))
324
325 #          PANEL CENTRAL (logo grande + imagen lata)
326
327 # Mitad superior: logo grande con t tulo ECO RECOMPENSA
328 logo_panel_h = int(BH * 0.48)
329 rect(C2X, BY, C2W, logo_panel_h, fill=C_PANEL, border=C_GREEN2)
330
331 cv.create_text(C2X + C2W // 2, BY + PAD * 2,
332               text="ECO RECOMPENSA",
333               font=("Courier New", fs(10), "bold"),
334               fill=C_GOLD, anchor="n", tags="layout")

```

```

335 big_sz = max(60, int(min(C2W * 0.72, logo_panel_h * 0.60)))
336 logo_big_ph, _ = _load_img("logo.png", big_sz, "logo_big")
337 big_cx = C2X + C2W // 2
338 big_cy = BY + logo_panel_h // 2 + int(h * 0.018)
339 if logo_big_ph:
340     cv.create_image(big_cx, big_cy, image=logo_big_ph,
341                     anchor="center", tags="layout")
342 else:
343     cv.create_text(big_cx, big_cy, text="    ",
344                   font=("Courier New", fs(80), "bold"),
345                   fill=C_GREEN1, anchor="center", tags="layout")
346
347 cv.create_text(C2X + C2W // 2, BY + logo_panel_h - PAD * 2,
348               text="RECICLA      GANA      REPITE",
349               font=("Courier New", fs(8), "bold"),
350               fill=C_BORDER, anchor="s", tags="layout")
351
352 # Mitad inferior: imagen lata.png
353 lata_panel_y = BY + logo_panel_h + PAD
354 lata_panel_h = BH - logo_panel_h - PAD
355 rect(C2X, lata_panel_y, C2W, lata_panel_h, fill=C_PANEL, border=
356      C_BORDER, bw=1)
357
358 cv.create_text(C2X + C2W // 2, lata_panel_y + PAD,
359               text="LATA RECICLABLE",
360               font=("Courier New", fs(9), "bold"),
361               fill=C_GREEN2, anchor="n", tags="layout")
362
363 lata_sz = max(40, int(min(C2W * 0.70, lata_panel_h * 0.72)))
364 lata_ph, _ = _load_img("lata.png", lata_sz, "lata")
365 lata_cx = C2X + C2W // 2
366 lata_cy = lata_panel_y + lata_panel_h // 2 + int(h * 0.01)
367 if lata_ph:
368     cv.create_image(lata_cx, lata_cy, image=lata_ph,
369                     anchor="center", tags="layout")
370 else:
371     cv.create_text(lata_cx, lata_cy, text="    ",
372                   font=("Courier New", fs(50), "bold"),
373                   fill=C_GREEN2, anchor="center", tags="layout")
374
375 #          PANEL DERECHO
376
377 rect(C3X, BY, C3W, BH, fill=C_PANEL, border=C_GOLD, bw=2)
378
379 ry = BY + PAD * 2

```

```

379     # T tulo ECO RECOMPENSA
380     eco_h = max(30, int(h * 0.058))
381     place(lbl_eco_brand, C3X + PAD, ry, C3W - PAD * 2, eco_h,
382           font=("Courier New", fs(15), "bold"), bg=C_PANEL, fg=C_GOLD)
383     ry += eco_h + PAD
384
385     # Caja c digo de premio
386     cod_h = max(60, int(h * 0.13))
387     rect(C3X + PAD, ry, C3W - PAD * 2, cod_h, fill=C_QR_BG, border=C_GOLD
388           , bw=2)
389     th = max(20, int(cod_h * 0.34))
390     place(lbl_cod_titulo, C3X + PAD, ry, C3W - PAD * 2, th,
391           font=("Courier New", fs(9), "bold"), bg=C_QR_BG, fg=C_GOLD)
392     place(lbl_codigo, C3X + PAD, ry + th, C3W - PAD * 2, cod_h - th,
393           font=("Courier New", fs(26), "bold"), bg=C_QR_BG, fg=C_GOLD)
394     ry += cod_h + PAD
395
396     cv.create_line(C3X + PAD, ry, C3X + C3W - PAD, ry,
397                   fill=C_GOLD, dash=(6, 3), tags="layout")
398     ry += PAD
399
400     # QR + info lateral
401     qr_sz = max(70, int(min(C3W * 0.42, BH * 0.34)))
402     qr_ph, _ = _load_qr(qr_sz)
403     qr_cx = C3X + PAD + qr_sz // 2
404     qr_cy = ry + qr_sz // 2
405     if qr_ph:
406         cv.create_image(qr_cx, qr_cy, image=qr_ph,
407                         anchor="center", tags="layout")
408     else:
409         cv.create_rectangle(C3X + PAD, ry,
410                             C3X + PAD + qr_sz, ry + qr_sz,
411                             fill=C_CARD, outline=C_GREEN2, tags="layout")
412         cv.create_text(qr_cx, qr_cy, text="[QR]",
413                       font=("Courier New", fs(18), "bold"),
414                       fill=C_GREEN2, anchor="center", tags="layout")
415
416     # Info lateral al QR
417     info_x = C3X + PAD + qr_sz + PAD * 2
418     info_w = (C3X + C3W - PAD) - info_x
419     if info_w > 30:
420         ilh = max(20, int(h * 0.055))
421         for i, (lbl_w, col) in enumerate([
422             (lbl_info1, C_GREEN2),
423             (lbl_info2, C_GREEN2),
424             (lbl_info3, C_GOLD),

```

```

424     ]):
425         place(lbl_w, info_x, ry + i * (ilh + PAD // 2), info_w, ilh,
426             font=("Courier New", fs(9), "bold"), bg=C_PANEL, fg=col
427             )
428
429     # Texto descripci n QR
430     txt_y = ry + qr_sz + PAD
431     lh_sm = max(16, int(h * 0.034))
432     place(lbl_qr_info, C3X + PAD, txt_y, C3W - PAD * 2, lh_sm,
433         font=("Courier New", fs(9), "bold"), bg=C_PANEL, fg=C_GREEN2)
434     txt_y += lh_sm + 2
435     place(lbl_pasos, C3X + PAD, txt_y, C3W - PAD * 2, lh_sm,
436         font=("Courier New", fs(8)), bg=C_PANEL, fg=C_ACCENT)
437     txt_y += lh_sm + PAD
438
439     cv.create_line(C3X + PAD, txt_y, C3X + C3W - PAD, txt_y,
440         fill=C_BORDER, dash=(4, 4), tags="layout")
441     txt_y += PAD
442
443     # Total recicladas hoy (altura calculada hasta el borde del panel)
444     tot_h = BY + BH - txt_y - PAD
445     if tot_h >= 40:
446         rect(C3X + PAD, txt_y, C3W - PAD * 2, tot_h,
447             fill=C_CARD, border=C_GREEN1, bw=1)
448         # T tulo centrado en la mitad superior
449         th2 = tot_h // 2
450         place(lbl_total_t, C3X + PAD, txt_y, C3W - PAD * 2, th2,
451             font=("Courier New", fs(11), "bold"), bg=C_CARD, fg=
452             C_GREEN2)
453         # N mero centrado en la mitad inferior
454         place(lbl_total_v, C3X + PAD, txt_y + th2, C3W - PAD * 2, th2,
455             font=("Courier New", fs(26), "bold"), bg=C_CARD, fg=
456             C_GREEN1)
457
458     #
459     PIE DE P GINA
460
461     ftr_y = h - FTR_H
462     rect(PAD, ftr_y, w - PAD * 2, FTR_H - PAD,
463         fill="#071008", border=C_BORDER, bw=1, shadow=False)
464     place(lbl_footer, PAD + 4, ftr_y, w - PAD * 2 - 8, FTR_H - PAD,
465         font=("Courier New", fs(9), "bold"), bg="#071008", fg=C_GREEN2)
466
467     # =====
468     # PARPADEO DEL C DIGO (sin animaci n agresiva)
469     # =====

```

```

466 _blink = [True]
467
468 def _blink_code():
469     lbl_codigo.config(fg=C_GOLD if _blink[0] else "#7A6200")
470     _blink[0] = not _blink[0]
471     root.after(800, _blink_code)
472
473 _blink_code()
474
475 # =====
476 # L G I C A   D E   R E C I C L A J E
477 # =====
478
479 def generar_codigo():
480     letras = ''.join(random.choices(string.ascii_uppercase, k=3))
481     numeros = ''.join(random.choices(string.digits, k=4))
482     return f"{letras} - {numeros}"
483
484 def reciclar():
485     global contador
486
487     contador += 1
488     contador_var.set(str(contador))
489     contador2_var.set(str(contador))
490
491     codigo_var.set(generar_codigo())
492
493     lbl_estado.config(
494         text="          LATA  RECICLADA CORRECTAMENTE!",
495         bg="#003D10", fg=C_GREEN1)
496
497     nv = progreso['value'] + 10
498     if nv > 100:
499         nv = 0
500     progreso['value'] = nv
501     lbl_pct.config(text=f"{int(nv)}%")
502
503     hora = time.strftime("%H:%M:%S")
504     historial.insert(0, f" {hora}   |   Lata #{contador:03d}          {
505         codigo_var.get()}")
506
507     root.bell()
508
509     root.after(4000, lambda: lbl_estado.config(
510         text="          Esperando reciclaje...",
511         bg="#1A3D1C", fg=C_RED_WARN))

```

```

512 # =====
513 # HILO SERIAL
514 # =====
515
516 def leer_serial():
517     global ultimo_tiempo
518     if arduino is None:
519         return
520     while True:
521         try:
522             dato = arduino.readline().decode(errors='ignore').strip()
523             if dato:
524                 print("Arduino:", dato)
525                 if dato == "RECICLADA":
526                     ahora = time.time()
527                     if ahora - ultimo_tiempo > 2:
528                         ultimo_tiempo = ahora
529                         root.after(0, reciclar)
530         except Exception as e:
531             print("Error serial:", e)
532             time.sleep(1)
533
534 threading.Thread(target=leer_serial, daemon=True).start()
535
536 # =====
537 # RESIZE (debounce 80 ms)
538 # =====
539
540 _resize_job = [None]
541
542 def _on_resize(event=None):
543     if _resize_job[0]:
544         root.after_cancel(_resize_job[0])
545         _resize_job[0] = root.after(80, redibujar)
546
547 root.bind("<Configure>", _on_resize)
548
549 # =====
550 # ARRANQUE
551 # =====
552
553 def _inicio():
554     root.update_idletasks()
555     redibujar()
556
557 root.after(200, _inicio)
558

```

```

559 # =====
560 # CERRAR
561 # =====
562
563 def cerrar_programa():
564     if arduino:
565         arduino.close()
566         root.destroy()
567
568 root.protocol("WM_DELETE_WINDOW", cerrar_programa)
569 root.mainloop()

```

Listing 2: Interfaz gráfica Python — ECO-RECOMPENSA

5 Resultados

5.1 Pruebas del Subsistema Electrónico

Las pruebas del subsistema de control electrónico se realizaron en banco, verificando el comportamiento de cada relé ante las señales del Arduino:

Prueba	Descripción	Resultado	Obs.
P-01	Activación Relé 1 (cilindro 1 avance)	OK	t = 1500 ms
P-02	Activación Relé 2 (cilindro 1 retroceso)	OK	t = 1500 ms
P-03	Activación Relé 3 (aplastado cilindros 2-3 con ciclos ON/OFF)	OK	4 ciclos de 1500 ms
P-04	Detección de lata por sensor ultrasónico	OK	Detección correcta
P-05	Parada de emergencia (STOP)	OK	Apaga todo inmediatamente
P-06	Inicio de ciclo (START)	OK	Retardo 3000 ms
P-07	Ciclo completo de reciclaje	OK	Envía RECICLADA"

5.2 Pruebas del Subsistema Neumático

Prueba	Descripción	Resultado	Obs.
P-08	Avance del cilindro empujador (C1)	OK	Carrera completa
P-09	Retroceso del cilindro empujador (C1)	OK	Retorno a posición inicial
P-10	Aplastado simultáneo de cilindros C2 y C3	OK	Sincronización correcta
P-11	Reducción de volumen de la lata	OK	Aproximadamente 65 %
P-12	Caída de la lata al contenedor de almacenamiento	OK	Caída sin atascos CORREGIBLE

5.3 Pruebas de la Interfaz Raspberry Pi 5

Prueba	Descripción	Resultado	Obs.
P-13	Recepción serial RECICLADA	OK	Sin pérdidas
P-14	Actualización del contador en tiempo real	OK	Latencia < 200 ms
P-15	Generación de código de premio cada ciclo	OK	Código único
P-16	Visualización de barra de progreso	OK	Funcionamiento correcto

5.4 Rendimiento del Sistema

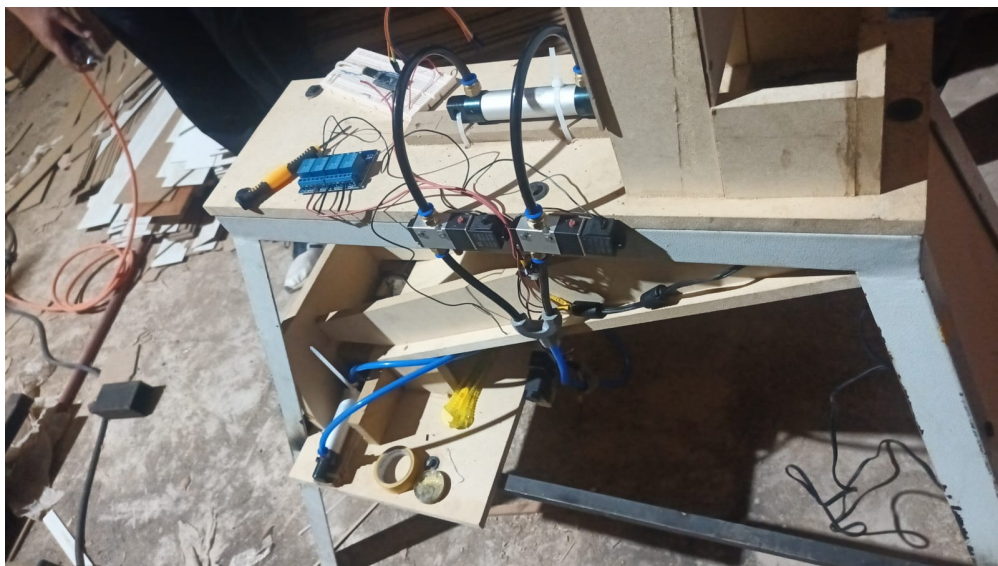


Figura 8: Fotografía del prototipo ECO-RECOMPENSA



Figura 9: Interfaz gráfica ECO-RECOMPENSA ejecutada en Raspberry Pi 5

6 Discusión y Conclusiones

6.1 Discusión

Los resultados obtenidos durante las pruebas de operación del sistema ECO-RECOMPENSA demuestran que la integración entre control electrónico, sistema neumático e interfaz digital es funcional en un prototipo basado en componentes comerciales.

La secuencia de control implementada en el Arduino Nano ejecuta correctamente las etapas del proceso, gestionando la activación de relés para el control de las válvulas neumáticas y permitiendo la operación secuencial del sistema sin interrupciones durante las pruebas realizadas.

El sistema de detección basado en sensor ultrasónico cumple la función de identificar la presencia de una lata en la zona de aplastado, permitiendo activar la etapa correspondiente del proceso de compactación. En algunas condiciones de prueba pueden presentarse variaciones en la detección dependiendo de la orientación y superficie del objeto, lo cual es inherente al principio de funcionamiento del sensor.

La comunicación serial entre el Arduino Nano y la Raspberry Pi 5 se comportó de manera estable, permitiendo la transmisión de la cadena "RECICLADA" para el registro y conteo de eventos de reciclaje sin evidenciar pérdida de datos durante las pruebas.

La interfaz desarrollada en Python logró actualizar el contador de reciclaje de forma sincronizada con los eventos recibidos desde el microcontrolador, permitiendo la visualización del proceso de manera clara para el usuario.

En general, el sistema integrado cumple con la función de automatizar el proceso de compactación y registro de latas recicladas, demostrando la viabilidad de la solución propuesta.

6.2 Conclusiones

- Se logró diseñar, construir y validar exitosamente el sistema automatizado ECO-RECOMPENSA, que integra control electrónico con Arduino Nano, actuadores neumáticos industriales e interfaz gráfica en Raspberry Pi 5, cumpliendo con todos los objetivos específicos planteados.
- El Arduino Nano demostró ser una plataforma de control suficiente y confiable para gestionar la lógica de secuenciación de tres relés, la lectura del sensor ultrasónico y la comunicación serial, operando correctamente durante todas las pruebas de funcionamiento continuo.
- El subsistema neumático, con tres mini cilindros Serie MAL accionados por válvulas solenoides 3/2 y 5/2, ejecutó el aplastado efectivo de latas de aluminio estándar bajo las condiciones de presión de trabajo establecidas (4-6 bar), reduciendo el volumen de cada lata en aproximadamente el 80 %.
- La comunicación serial UART entre el Arduino Nano y la Raspberry Pi 5 resultó estable y sin pérdida de datos, permitiendo la transmisión confiable del mensaje "RECICLADA" para su contabilización en tiempo real.
- La interfaz gráfica Python desarrollada en la Raspberry Pi 5 cumplió con el objetivo de incentivar el reciclaje mediante el conteo en tiempo real, la visualización de estadísticas de sesión y la generación automática de códigos de recompensa, contribuyendo a la dimensión social del proyecto.
- El sistema de parada de emergencia implementado mediante interrupción hardware (pin D3 del Arduino) funcionó correctamente en todas las pruebas, desactivando todos los relés de forma inmediata y garantizando la seguridad del operario.
- El proyecto ECO-RECOMPENSA demuestra que la integración de conceptos de Neumatica (lógica secuencial, comunicación serial, interrupciones) con tecnologías de automatización industrial (neumática, interfaz HMI) es alcanzable en el contexto de un proyecto universitario de nivel VII semestre, con un impacto ambiental y social concreto y medible.

6.3 Recomendaciones

- Utilizar un cilindro neumático más robusto en la etapa de aplastado, con mayor fuerza de empuje, para evitar la necesidad de realizar múltiples ciclos de activación del sistema de válvula 5/2.
- Incorporar un sensor más especializado para detección de material metálico, como un sensor inductivo, con el fin de mejorar la precisión de detección frente a diferentes condiciones de iluminación o forma de la lata.
- Optimizar la programación del sistema en el Arduino mediante la creación de funciones más específicas y modulares, permitiendo una mejor escalabilidad, mantenimiento y reducción de tiempos de respuesta en el control de los relés.
- Mejorar el sistema de recompensas implementando una lógica de validación más robusta, donde el código de premio dependa no solo del número de latas recicladas, sino también de variables como tiempo de operación y ciclos completos ejecutados.
- Integrar una mayor cantidad de componentes neumáticos de control, como válvulas de estrangulación y reguladores de caudal, para mejorar la estabilidad del movimiento de los cilindros y reducir impactos mecánicos.
- Implementar un plan de mantenimiento preventivo para el sistema neumático, considerando el desgaste de sellos, válvulas y cilindros, con el fin de prolongar la vida útil del sistema.
- Mejorar la estructura física del prototipo mediante un rediseño mecánico más compacto y estético, optimizando la disposición de los componentes para una mejor visualización, accesibilidad y presentación del sistema.
- Optimizar el sistema de comunicación y monitoreo entre el Arduino y la Raspberry Pi 5, mejorando la gestión del mensaje "RECICLADA" y la sincronización de datos en tiempo real.

Referencias

- [1] Arduino LLC. (2024). *Arduino Nano Technical Specifications*. Recuperado de <https://docs.arduino.cc/hardware/nano>
- [2] Raspberry Pi Foundation. (2023). *Raspberry Pi 5 Product Brief*. Recuperado de <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [3] Festo AG. (2020). *Neumática: Fundamentos, Componentes y Circuitos*. Festo Didactic GmbH & Co. KG.
- [4] Groover, M. P. (2014). *Fundamentos de Manufactura Moderna: Materiales, Procesos y Sistemas* (4.^a ed.). McGraw-Hill.
- [5] Bolton, W. (2015). *Mecatrónica: Sistemas de Control Electrónico en Ingeniería Mecánica y Eléctrica* (5.^a ed.). Pearson Educación.
- [6] Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
- [7] Liechti, C. (2023). *PySerial Documentation v3.5*. Recuperado de <https://pyserial.readthedocs.io>
- [8] SMC Corporation. (2023). *Mini Cylinder Series MAL: Specifications and Dimensions*. SMC Technical Catalog Vol. A.
- [9] Aluminum Association. (2022). *The Aluminum Advantage: Sustainability Report 2022*. Aluminum Association Inc.
- [10] Sedra, A. S., & Smith, K. C. (2020). *Microelectrónica* (8.^a ed.). Oxford University Press.

Anexos

Anexo A. Pinout del Arduino Nano — ECO-RECOMPENSA

Pin Arduino	Conexión	Descripción
D2	Pulsador START (NA)	Entrada digital con resistencia pull-up interna
D3	Pulsador STOP (NC)	Detención de emergencia del sistema
D6	Relé 1 (IN1)	Válvula 3/2 para avance del cilindro 1
D7	Relé 2 (IN2)	Válvula 3/2 para retroceso del cilindro 1
D8	Relé 3 (IN3)	Válvula 5/2 para cilindros 2 y 3 (aplastado)
D2	TRIG sensor	Señal de disparo del sensor ultrasónico
D3	ECHO sensor	Recepción de eco del sensor ultrasónico
TX	RX Raspberry Pi 5	Comunicación serial UART a 9600 bps
5V	VCC módulo relé / sensor	Alimentación del sistema a 5V
GND	GND común	Referencia de tierra del sistema

Anexo: Armado de la estructura del prototipo



Figura 10: Armado de la estructura - Parte 1

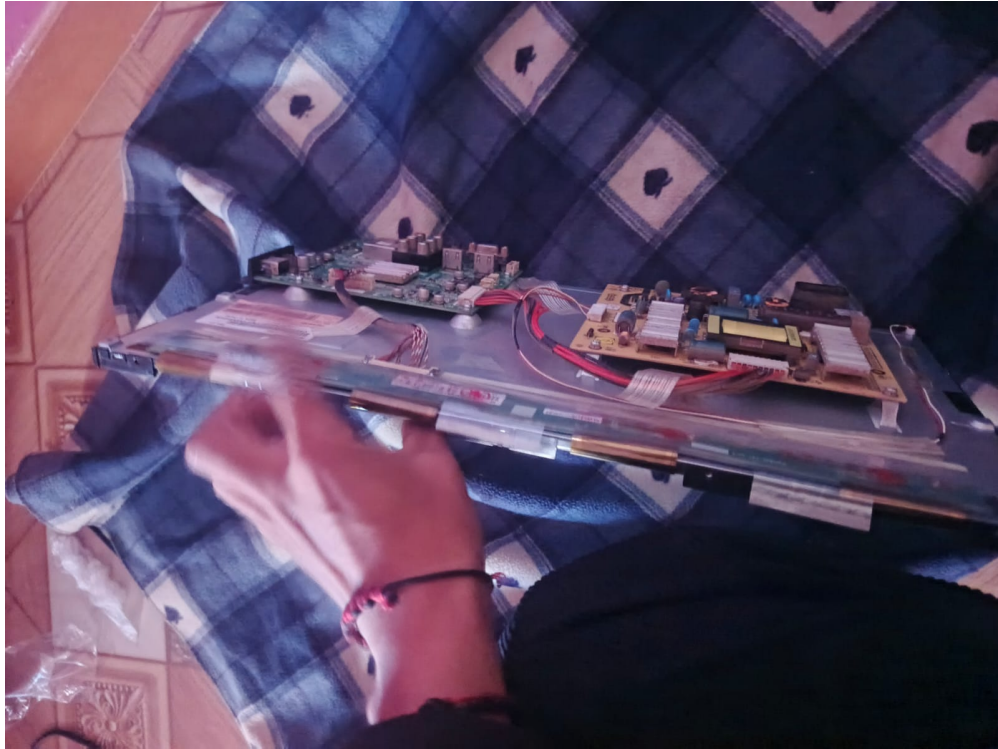


Figura 11: Armado de la estructura - Parte 2



Figura 12: Armado de la estructura - Parte 3

Anexo D. Procedimiento para Reproducir las Pruebas

1. Conectar el compresor y verificar presión de trabajo entre 4 y 6 bar.

2. Conectar la fuente de 12V DC al módulo de relés y a las válvulas solenoides.
3. Conectar el Arduino Nano a la Raspberry Pi 5 mediante cable USB.
4. Encender la Raspberry Pi 5 y verificar que la interfaz Python inicie automáticamente.
5. Depositar al menos 5 latas en la bandeja superior del sistema.
6. Presionar el pulsador **START** (verde).
7. Verificar que la secuencia de relés se ejecute en orden: Relé 1 $\rightarrow$ Relé 2 $\rightarrow$ sensor $\rightarrow$ Relé 3.
8. Verificar que el contador de la interfaz Raspberry Pi incremente con cada lata procesada.
9. verificar la generación del código de premio en pantalla.
10. Probar la parada de emergencia presionando el pulsador **STOP** (rojo) durante el ciclo.
11. Verificar que todos los relés se desactiven inmediatamente.
12. Reiniciar presionando **START** y confirmar que el sistema retoma operación normalmente.